SC2002 Object-Oriented Design & Programming

Restaurant Rush

Group Assignment Report - Design, Diagrams and Evidence

Item	Details (fill in before submitting)
Lab group / Group	[e.g. FCS1 / Grp1]
Members	[Name 1 - Matric] [Name 2 - Matric] [Name 3 - Matric] [Name 4 - Matric]
GitHub repository	[https://github.com/<team>/restaurant-rush]
Tags	`stage-1`, `stage-2`, `stage-3` (stage-3 = final submission)
Language / tools	Java (compiled with JDK 21, source compatible with Java 11+), no external libraries; PlantUML for diagrams

Contents. 1 Overview and how to run - 2 Diagrams (class, sequence) - 3 Design discussion - 4 Functional evidence - 5 Stage comparisons - 6 Reflections - Appendix: interpretations of the brief.

1 Overview and how to run

Restaurant Rush is a deterministic, turn-based command-line game. The player is the manager of a restaurant with two tables and runs a Host, a Waiter and a Chef each turn (the Cashier works automatically) for 18 turns. Six customers arrive on turns 1, 4, 7, 10, 13 and 16. The game is won if, after turn 18, at least 4 customers have paid, revenue is at least $45.00 and the average recorded satisfaction of paid customers is at least 60.

The game was built in three stages, each a runnable version marked by a git tag. Every stage is a complete game; later stages add behaviour without removing earlier behaviour.

Tag	What it adds	Customers in the reproducible run
`stage-1`	Regular customers, menu (Salad $9 / Burger $15 / Pasta $18), Waiter, Chef, automatic Cashier, 2 tables, queue, six-step turn loop, victory/defeat	6 Regular
`stage-2`	VIPCustomer (loses 5/turn, 10% discount) and CriticCustomer (loses 12/turn, always orders Pasta, publishes a review)	Regular, Critic, VIP, Regular, Regular, Regular
`stage-3`	Host (seats one extra customer per turn), ComboMeal (two items at 90% of the combined price), Happy Hour (once per game, 2 turns, 20% off new orders, +15 satisfaction to unserved customers at turn start)	same as Stage 2

Build and run

```
./build.sh                       # Windows: build.bat   (needs JDK 11 or newer)
./run.sh                         # Windows: run.bat     - play interactively
./run.sh < runs/stage3_victory.txt   # replay a saved, reproducible game
```

Commands are typed one line at a time: `seat <C#>`, `order <C#>`, `serve <order#>`, `cook <order#>`, `wait`, and in Stage 3 also `happyhour`. Anything the rules do not allow prints a reason and asks again without using up the staff member's task. Source is organised as `rr.domain` (rules and entities), `rr.control` (turn engine) and `rr.ui` (console only).

2 Diagrams

2.1 Class diagram (final design, Stage 3), annotated

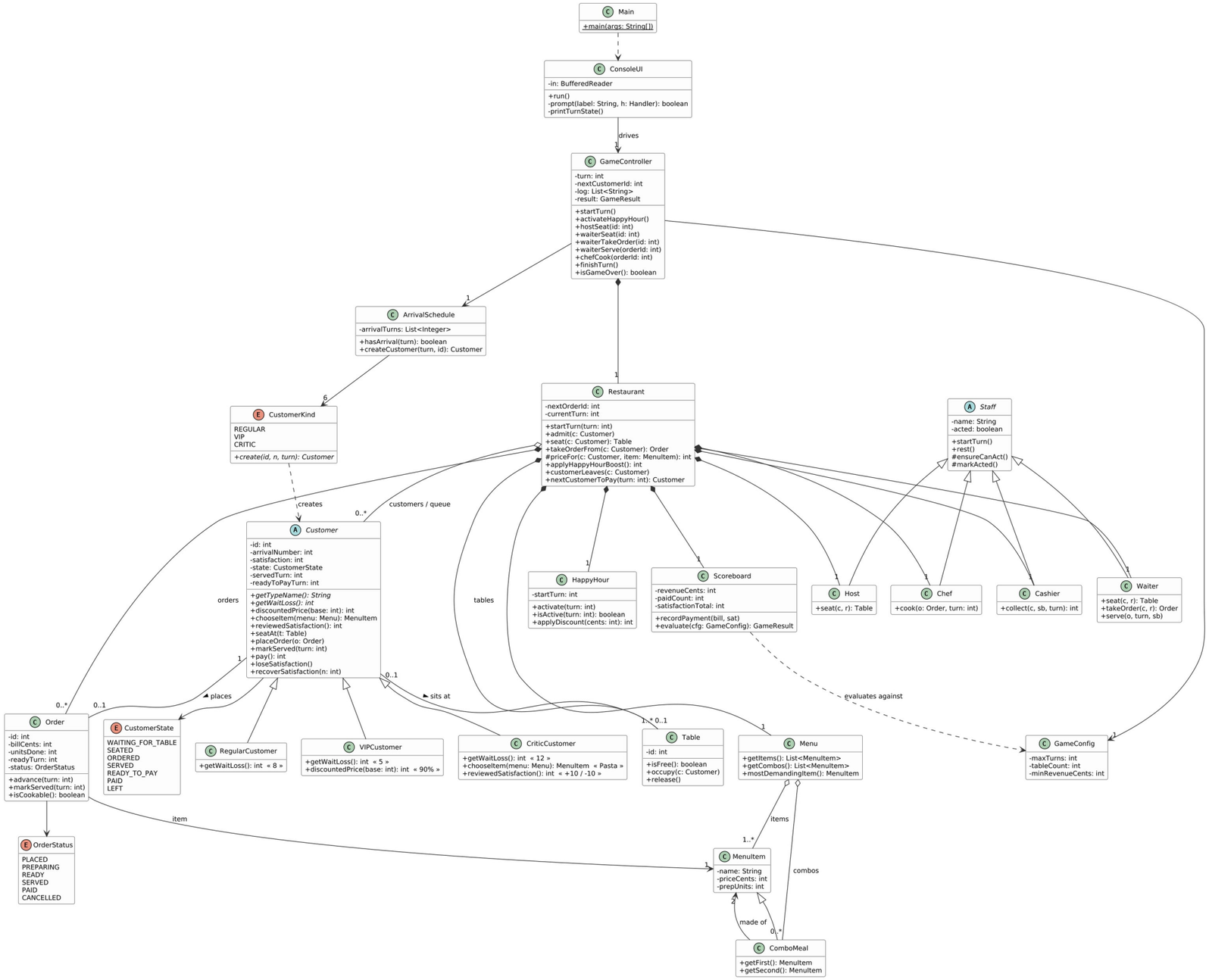


Figure 1 - Class diagram of the Stage 3 code. Visibility: + public, - private, # protected. Multiplicities are on the associations.

Encapsulation

`Customer.state` and `satisfaction` are private. The only way to change them is `seatAt`, `placeOrder`, `markServed`, `pay`, `loseSatisfaction`; each checks its own precondition and throws `InvalidActionException` (e.g. `placeOrder` refuses unless the customer is SEATED). `Order.billCents` is final, so the price is locked when the order is taken and Happy Hour can never change it afterwards. `Staff.acted` is private and only reachable through `ensureCanAct()` / `markActed()`.

Polymorphism

`GameController.finishTurn()` calls `loseSatisfaction()`, and `Restaurant.takeOrderFrom()` calls `chooseItem()` and `discountedPrice()`, on a `Customer` without ever asking what kind it is. `RegularCustomer`, `VIPCustomer` and `CriticCustomer` override `getWaitLoss()`; VIP overrides `discountedPrice()`; Critic overrides `chooseItem()` and `reviewedSatisfaction()`. `ComboMeal` is used wherever a `MenuItem` is expected, so `Order`, `Chef` and the bill calculation needed no changes. Each `CustomerKind` constant overrides `create()`.

Composition and aggregation

Composition (filled diamond): `Restaurant` creates and owns its `Tables`, `Orders`, `Waiter`, `Host`, `Chef`, `Cashier`, `Menu`, `HappyHour` and `Scoreboard`; `GameController` owns the `Restaurant`. **Aggregation** (hollow diamond): a `Restaurant` holds `Customers` that are created elsewhere (by the `ArrivalSchedule`); `Menu` holds `MenuItems`.

Inheritance and abstraction

`Customer` has 3 subclasses, `Staff` has 4, `MenuItem` has `ComboMeal`. `Customer` and `Staff` are abstract: `Customer` leaves `getTypeName()` and `getWaitLoss()` abstract, so a new kind cannot forget to say how fast it loses patience.

2.2 Sequence diagram

Scenario. Turn 7 of the Stage 3 reproducible run (`runs/stage3_victory.txt`): the VIP customer c3 arrives; the manager decides whether to start Happy Hour; the Host seats c3, the Waiter takes the order, the Chef cooks it over two turns; the dish becomes READY, is SERVED, c3 eats, pays, and the game ends with the result check. The manager's choice that changes the outcome is shown as the `alt` block: with Happy Hour the VIP's Pasta is billed $12.96 (= $18.00 x 0.9 x 0.8) and c3 recovers 15 satisfaction twice; without it the bill is $16.20 and c3 is recorded at 90 instead of 95 (see section 3.6). Behaviour from Stages 2 and 3 appears in `discountedPrice()` (VIP override), `HappyHour.applyDiscount()` and the Host.

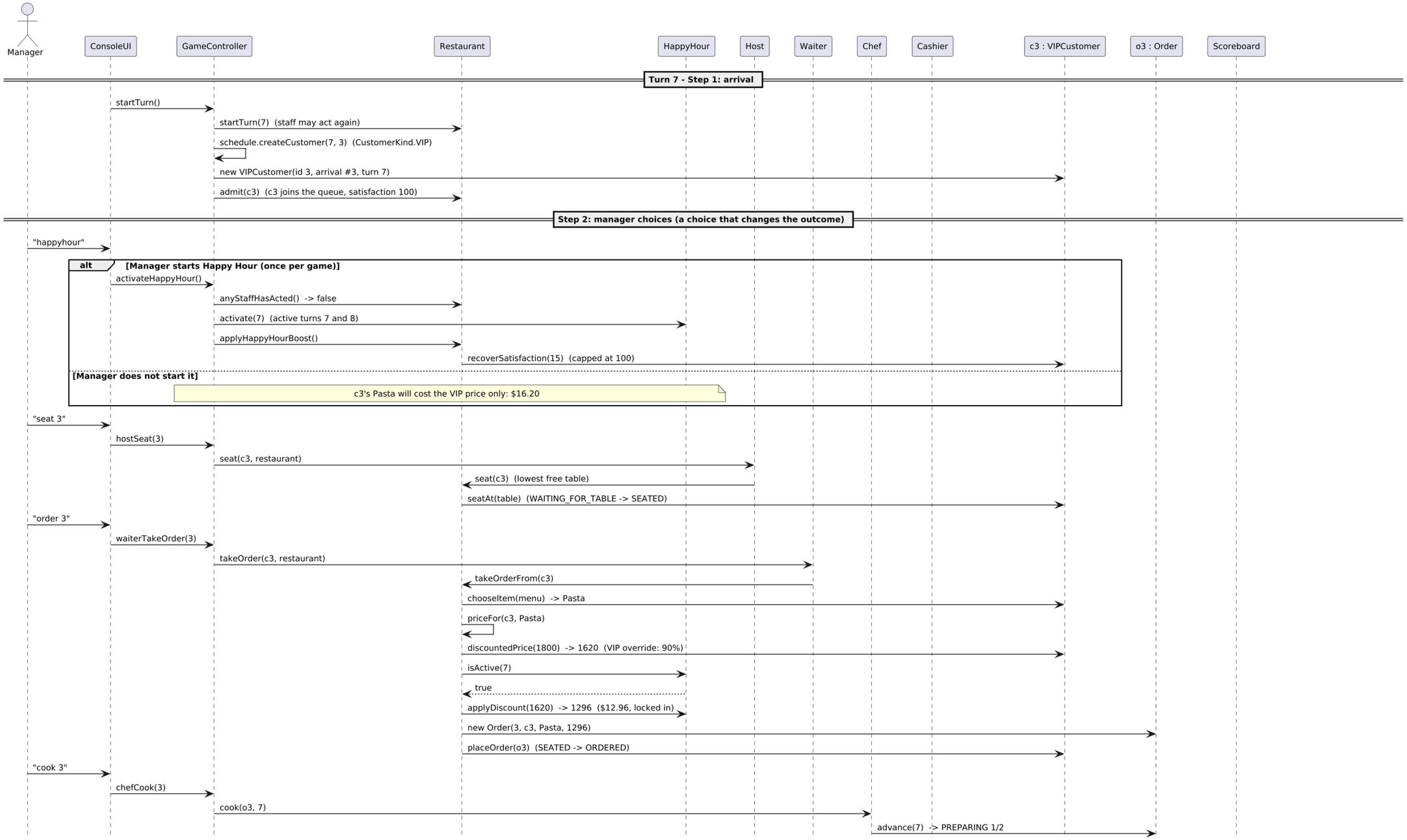


Figure 2a - Sequence diagram, part 1: turn 7, steps 1-3 (arrival, manager choices, order and cooking).

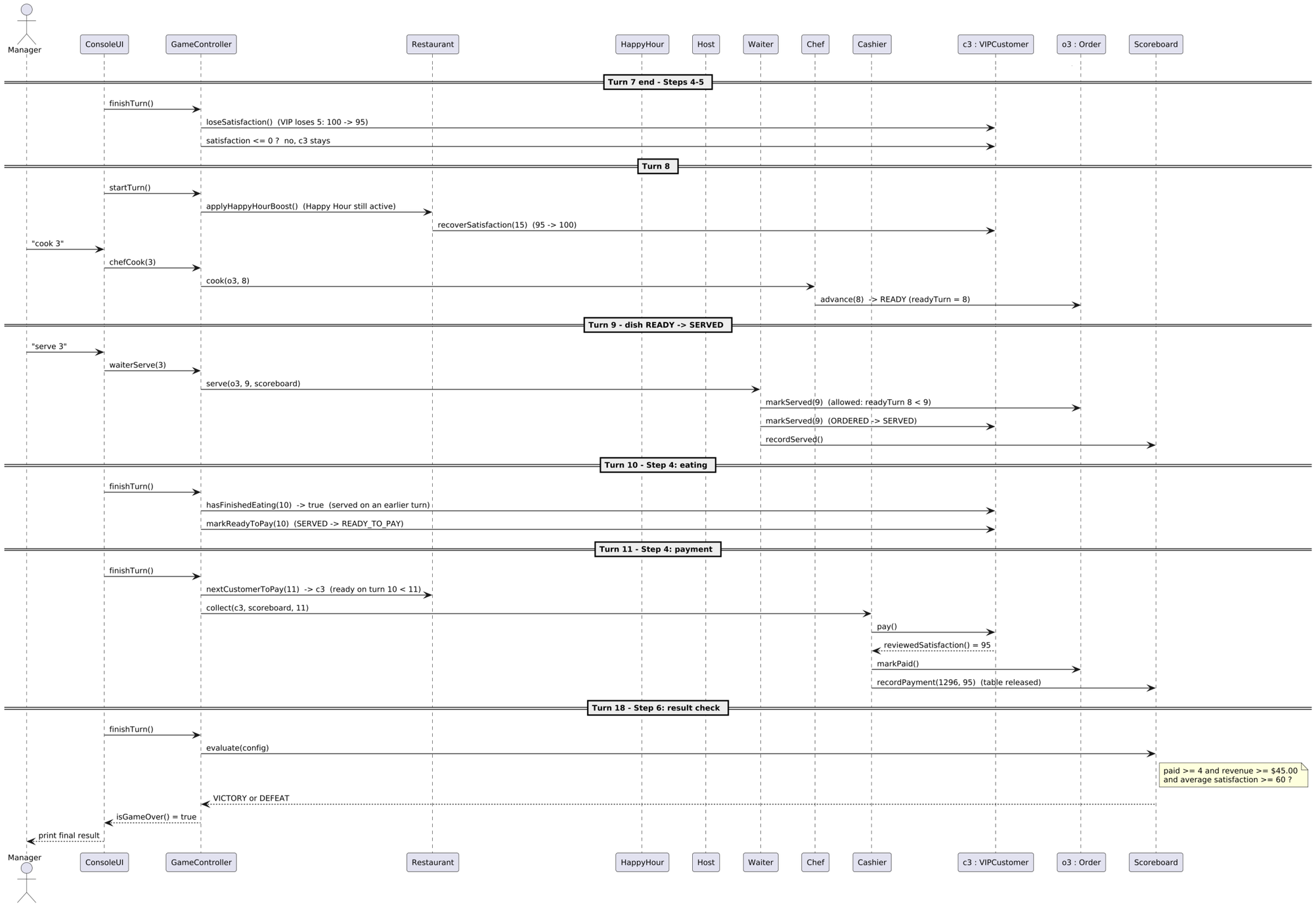


Figure 2b - Sequence diagram, part 2: turn 7 end (satisfaction loss), turn 8 (READY), turn 9 (SERVED), turn 10 (eating), turn 11 (payment), turn 18 (result check).

3 Design discussion

3.1 Abstractions and responsibilities

Class	Responsibility	Why it is its own class
`ConsoleUI`	Reads commands, prints state, shows rule errors. No rules.	The rules can be tested or reused with another front end.
`GameController`	Turn counter and the six turn steps in the required order; collects an event log for the UI.	One place that owns "what happens when".
`Restaurant`	Owns tables, queue, orders, staff, menu, Happy Hour, scoreboard. Seats customers, takes orders, calculates the locked bill, handles walkouts.	Aggregate root: everything that must stay consistent (table occupancy, queue, orders) lives behind one API.
`Customer` (+ `RegularCustomer`, `VIPCustomer`, `CriticCustomer`)	State of one visit and the behaviours that differ by kind: wait loss, discount, food choice, review.	The variation point of Stage 2. Subclasses override hooks, nothing else changes.
`CustomerKind`	Builds the right customer for each schedule entry.	Removes `if/switch` on kind from the schedule and controller.
`Order`, `OrderStatus`	Prep progress, status life cycle, locked bill, ready turn.	Enforces "READY this turn can only be served next turn".
`Menu`, `MenuItem`, `ComboMeal`	What can be ordered, price, prep units; combo = 90% of two items.	Stage 3 added `ComboMeal` by subclassing, not by editing `Order` or `Chef`.
`Staff`, `Waiter`, `Host`, `Chef`, `Cashier`	One task per turn (`ensureCanAct`, `markActed`, `rest`) plus the role-specific operation.	Makes staff time the scarce resource the player manages.
`Table`	Free/occupied.	Small, but gives seating a real object to check.
`HappyHour`	One-time two-turn window, discount, boost constants.	Keeps the feature self-contained and removable.
`Scoreboard`	Revenue, paid count, satisfaction total, walkouts; decides victory.	The win rule changes in one place.
`ArrivalSchedule`, `GameConfig`	Scenario data: who arrives when, table count, targets.	Changing the scenario does not touch game logic.
`InvalidActionException`, `Money`	Rule violations carry a player-readable message; money is integer cents with half-up rounding.	No floating-point money errors ($10.80 is exactly 1080).

3.2 SOLID: where it applies, and where it does not

Principle	Evidence in this code	Honest limits
Single responsibility	`ConsoleUI` only does I/O; `Scoreboard` only does the numbers; `HappyHour` only knows its window; `Order` only tracks one order.	`Restaurant` has several jobs (seating, ordering, pricing, queries). We accepted this to keep invariants in one place (see 3.4).
Open/closed	Stage 1 to 2 added three classes and changed three small files (`ArrivalSchedule`, `Menu`, `Main`). `GameController`, `Restaurant`, `Customer`, the staff and the UI were not touched. `ComboMeal` extends `MenuItem` without editing `Order`/`Chef`.	Stage 2 to 3 did modify existing classes (`Restaurant`, `GameController`, `ConsoleUI`, `Customer`, `Menu`) because Host and Happy Hour are new rules about the restaurant, not new kinds of an existing thing.
Liskov substitution	Every `Customer` subclass keeps the contract: wait loss >= 0, discounted price <= list price, reviewed satisfaction stays in 0..100. `ComboMeal` behaves as a `MenuItem`.	`Host` extends `Staff` but is not a `Waiter` on purpose; making it a subclass of `Waiter` would let it take orders.

Principle	Evidence in this code	Honest limits
Interface segregation	Not claimed. We created no interfaces just to tick a box; the only interface is the private one-method `Handler` in the UI.	With more roles the staff operations might deserve small interfaces.
Dependency inversion	Game logic depends on the abstract `Customer` and on `MenuItem`, not on concrete kinds.	The controller still depends on concrete `Restaurant` and staff classes. We did not add interfaces there because there is only one implementation.

3.3 Turn engine and when choices are applied

`GameController` maps the six required steps to code: **step 1** `startTurn()` (new arrival, staff task counters reset, Happy Hour boost if active); **steps 2-3** `hostSeat/waiterXxx/chefXxx`; **step 4** the first half of `finishTurn()` (Cashier takes the oldest customer who was READY_TO_PAY on an earlier turn, then customers served on an earlier turn finish eating); **step 5** the second half (unserved customers lose satisfaction, at 0 they leave, their order is cancelled and the table is freed); **step 6** the result check after turn 18.

The brief separates "manager choices" (step 2) from "kitchen and serving" (step 3). We validate and apply each choice at the moment it is entered, always in the order Host, Waiter, Chef. This gives exactly the required results: the Waiter always acts before the Chef, so a dish that becomes READY this turn can never be served this turn (and `Order.markServed` additionally checks `readyTurn < turn`); the Chef can work on an order the Waiter took this turn; an order taken this turn is billed with this turn's Happy Hour status. Applying choices immediately also lets the UI reject an invalid choice on the spot and ask again without consuming the task.

Happy Hour timing. `happyhour` is only accepted before any staff member has acted that turn (`Restaurant.anyStaffHasActed()`), once per game. It is active on the turn it is started and the next one. The boost is applied on activation and again at the start of the next turn; prices are locked at order time, so Happy Hour never changes orders placed earlier.

3.4 Trade-offs we made

- **Fat Restaurant class.** Putting seating, ordering and pricing in the aggregate root keeps table/queue/order invariants in one place and made `GameController` simple, but `Restaurant` is the class most likely to grow. Splitting out a `Pricing` class would be the next refactor.
- **Thin staff classes.** `Chef.cook` is only a few lines. Their real job is modelling "one task per turn" and giving Stage 3 a natural place to add `Host`. We preferred that to an over-designed task framework.
- **Deterministic food choice.** Food is chosen by a documented rule (arrival number cycles Salad, Burger, Pasta; in Stage 3 even arrivals take a combo; Critic always Pasta) instead of random numbers, so every run is reproducible and the manager can plan. The cost is less variety.
- **Enum for customer state, not the State pattern.** `CustomerState` plus guard checks in `Customer` is shorter and clear enough for seven states; separate state classes would add files without removing any branching we actually have.
- **Combo rule inside the customer's food choice.** It is a small edit to an existing class (Stage 3) and makes `Customer` know that menus can have combos. We accepted it rather than add a separate "ordering policy" class for one rule.
- **Integer cents.** All money is `int` cents; the only rounding is `Money.percentOf` (half up), applied in the required order VIP then Happy Hour.

3.5 An alternative design we considered

We considered modelling every manager choice as a **Command object** (`SeatCommand`, `TakeOrderCommand`, `CookCommand`, ...). The UI would queue a command for each staff member and `GameController` would execute them in step 3 in the order Waiter then Chef. Advantages: a clean separation of step 2 (choose) from step 3 (resolve), easy logging and an undo option. We rejected it because the Chef's valid choices depend on what the Waiter does in the same turn (an order taken this turn exists only after the Waiter's command has run), so every command would need to be validated against a "projected" future state, which duplicates rules and adds many classes. Applying choices immediately in a fixed role order produced identical game outcomes with far less code. If the game grew to many simultaneous actors, the Command design would become worth it.

3.6 How the manager's choices change the result

Because the Waiter and Host have one task per turn, ordering matters. All figures below come from saved, reproducible runs (section 4):

Choice	Run A	Run B	Effect
Turn 10 (Stage 1): serve the finished Pasta first, or seat the new arrival first?	Serve first (`stage1_victory`) C3 recorded 76, average 79.2	Seat first (`stage1_variant_seat_first`) C3 recorded 68, average 77.6	The customer who has already waited longest keeps losing 8 points per turn. Seating the newcomer first gains nothing because the newcomer is not ordering until the next turn anyway.
Turn 7 (Stage 3): start Happy Hour for the VIP's order?	With (`stage3_victory`) bill $12.96, C3 recorded 95 revenue $76.56, avg 89.8	Without (`stage3_no_happyhour`) bill $16.20, C3 recorded 90 revenue $79.80, avg 88.8	Happy Hour trades $3.24 of revenue for +5 satisfaction on that customer. Both win; in a tighter game the manager would pick based on which target is at risk.
Do nothing at all	`stage1_defeat`, `stage2_defeat`, `stage3_defeat`	-	0 paid; customers walk out once satisfaction hits 0 (the Critic after 9 unserved turns, a Regular after 13) -> DEFEAT.

4 Functional evidence

Every run is replayable: `./run.sh < runs/<script>.txt` prints the same transcript (saved as `runs/<script>.out.txt`). Stage 1 and Stage 2 scripts need their own tag checked out (`git checkout stage-1`), because Stage 3 added the Host prompt.

Tag	Script	Result
stage-1	`stage1_victory`	5 paid, $66.00, average satisfaction 79.2 - VICTORY
stage-1	`stage1_defeat` (everyone waits)	0 paid, $0.00, 2 customers walked out - DEFEAT
stage-1	`stage1_invalid_demo`	Unknown command, missing number, unknown customer, order before seating, cook with no order, serve before READY: each prints a reason and the same staff member is asked again
stage-1	`stage1_variant_seat_first`	5 paid, $66.00, average 77.6 - victory, but lower than the best line (section 3.6)
stage-2	`stage2_victory` (Critic + VIP)	5 paid, $67.20 (VIP Pasta $16.20), average 78.0 - VICTORY. Critic served at 64 is recorded as 74 (+10 review).
stage-2	`stage2_defeat`	Critic leaves at the end of turn 12, a Regular at turn 13, 0

Tag	Script	Result
		paid - DEFEAT
stage-3	`stage3_victory` (Host, combo, Happy Hour)	5 paid, $76.56, average 89.8 - VICTORY. VIP Pasta $12.96; C4 orders the combo "Burger + Salad" ($21.60, 2 prep units).
stage-3	`stage3_no_happyhour`	5 paid, $79.80, average 88.8 - VICTORY
stage-3	`stage3_defeat`	0 paid - DEFEAT

Excerpt (Stage 1, invalid choices are explained and the task is not consumed):

```
Waiter> serve 1
  [Not allowed] There is no order #1. Choose again.
Waiter> order 1
  [Not allowed] Customer C1 is not seated and waiting to order (currently
WAITING_FOR_TABLE). Choose again.
Waiter> seat 1
  > Waiter seated C1 (Regular) at T1.
```

Excerpt (Stage 3, stacked discount):

```
Host> happyhour
  > HAPPY HOUR started (this turn and next): orders taken cost 80% of their price; 1
unserved customer(s) recover 15 satisfaction.
  > Waiter took C3's order. C3 chose Pasta (order #3, bill $12.96, 2 prep unit(s)).
```

Requirement coverage

Requirement from the brief	Where it is met
18 turns, 2 tables, 6 scheduled arrivals	`GameConfig.base()`, `ArrivalSchedule`
One task per staff member per turn; waiting uses the task	`Staff.ensureCanAct/markActed/rest`
Invalid choice: explain, choose again, task not consumed	`InvalidActionException` thrown before `markActed()`; `ConsoleUI.prompt` loops
READY this turn can only be served next turn	`Order.markServed` (`readyTurn < turn`); Waiter acts before Chef
Eating takes a full turn; Cashier pays oldest first, one per turn, never in the turn the customer became ready	`Customer.hasFinishedEating`, `Restaurant.nextCustomerToPay`, `Cashier.collect`
Satisfaction loss for unserved customers; at 0 leave, cancel order, free table	`GameController.finishTurn` step 5, `Restaurant.customerLeaves`
Victory: >= 4 paid, >= $45.00, average >= 60	`Scoreboard.evaluate` (integer comparison, no rounding)
Customer kinds without kind checks in the controller	`getWaitLoss`, `discountedPrice`, `chooseItem`, `reviewedSatisfaction` overrides
Pricing order VIP then Happy Hour, locked at order time	`Restaurant.priceFor`, `Order.billCents` final
Money in integer cents	`Money`, all `*Cents` fields
State, available tasks and results shown every turn	`ConsoleUI.printTurnState`, `printScoreLine`, `printResult`

5 Stage comparisons

Repository: [`https://github.com/<team>/restaurant-rush`]. Compare links (replace the placeholder with your repository): `.../compare/stage-1...stage-2` and

`.../compare/stage-2...stage-3`. The same information locally: `git diff --stat stage-1 stage-2`.

5.1 Stage 1 to Stage 2

Change	File	Reason
New	`VIPCustomer`, `CriticCustomer`	The two new customer kinds, overriding the existing hooks.
New	`CustomerKind`	Lets the schedule create any kind without `if/switch`.
Changed	`ArrivalSchedule`	Was hard-wired to Regular; now takes a list of kinds and offers `base()` (Stage 1) and `mixed()` (Stage 2).
Changed	`Menu`	Added `mostDemandingItem()` for the Critic's order rule.
Changed	`Main`	Uses the mixed schedule.
Deleted	none	-
Untouched	`GameController`, `Restaurant`, `Customer`, staff, `ConsoleUI`	The new behaviour arrives through overridden methods.

5.2 Stage 2 to Stage 3

Change	File	Reason
New	`ComboMeal`, `Host`, `HappyHour`	The three new concepts, each self-contained.
Changed	`Customer`	`chooseItem` also handles combos; `recoverSatisfaction` for the Happy Hour boost.
Changed	`Menu`	Holds a list of combos.
Changed	`Restaurant`	Owns the Host and Happy Hour; `startTurn(turn)` knows the current turn; `priceFor` applies VIP then Happy Hour; boost and "has any staff acted" helpers.
Changed	`GameController`	`hostSeat`, `hostWait`, `activateHappyHour`, start-of-turn Happy Hour boost.
Changed	`ConsoleUI`	Host prompt, `happyhour` command, Happy Hour status line.
Deleted	none (a few lines were replaced)	-

Local commits: `b8b5561` (stage-1), `087630e` (stage-2), `a2f5118` (stage-3); the final commit on `main` adds the report, diagrams and documentation only.

6 Reflections

- **What worked.** Making the points where customers differ into overridable methods meant Stage 2 touched no game-flow code at all. Writing the saved input scripts early let us check each stage with exact expected numbers.
- **What we would change.** `Restaurant` grew the most in Stage 3 and is the weakest part of the design; a separate pricing class and a separate seating/queue class would be the next refactor. The Host being a distinct prompt each turn makes the UI slightly repetitive.
- **Surprise.** Satisfaction rules interact: the Critic's review can turn a good result into a bad one if the Critic is served late, which makes the "who do I serve first" decision more meaningful than the raw numbers suggest.
- **Testing approach.** All tests are deterministic replays of saved scripts compared against hand-calculated expectations (for example VIP Pasta in Happy Hour: 1800 x 0.9 x 0.8 = 1296 cents). We did not use a unit-test framework.

Appendix: interpretations of the brief

- The Waiter and Host seat the queued customer the manager names, at the lowest-numbered free table.
- The Waiter may take an order from a customer the Host seated on the same turn, mirroring "the Chef may work on an order the Waiter took this turn".
- "Waiting for food" for Happy Hour and for satisfaction loss means any customer who is queued, seated or ordered but not yet served.
- Critic special behaviour (our design): always orders the dish with most prep units (Pasta); the satisfaction recorded at payment is a review: +10 if at least 60, -10 if below 60 (kept within 0..100).
- Combo rule: even-numbered arrivals order combo number (n/2 - 1) mod 2 from [Salad + Pasta, Burger + Salad]; the Critic still orders Pasta.
- Average satisfaction is compared as `total >= 60 x paid` (integers), so no rounding can change the result.